

BASH SCRIPTING FUNDAMENTALS

Shell Basics

Terminal navigation, commands, PATH, and your first scripts

Merit Advisory
Bash Scripting Fundamentals Bootcamp
Beginner-friendly lecture materials
July 2026

July 2026



Agenda

- 1 Section 1: Shell and Terminal Foundations — 4 slides**
- 2 Section 2: Navigation and Paths — 4 slides**
- 3 Section 3: Listing and Inspecting Directories — 4 slides**
- 4 Section 4: Creating and Moving Files — 4 slides**
- 5 Section 5: Removing and Viewing Text — 4 slides**
- 6 Section 6: Finding Commands and Getting Help — 4 slides**
- 7 Section 7: Output and First Scripts — 4 slides**
- 8 Section 8: Environment and Identity — 4 slides**

9 Section 9: System Awareness and Patterns — 4 slides

1 Section 10: Safe Command Habits — 4 slides

0

1 Section 11: Keyboard Workflow — 4 slides

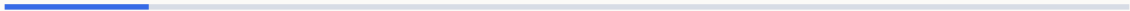
1

1 Section 12: Practice Lab and Checkpoints — 4 slides

2

1 Section 1

Shell and Terminal Foundations



1.1 What Is a Shell · What Is Bash

- A **shell** is a program that accepts commands and asks the operating system to run them.
- It gives you a text-based way to open files, start programs, and inspect the computer.
- Learning the **shell** helps you automate repeated work instead of clicking through menus.
- **Bash** is a common shell used on Linux, macOS, and many server environments.
- It can run commands interactively and also read commands from script files.
- Many automation examples use **Bash** because it is widely available and well documented.

Examples

```
# Print a message
echo "Hello from the shell"
# Run a simple command
date
# Show Bash path
command -v bash
# Show Bash version
bash --version
```

1.2 Terminal Versus Shell · Prompt

- A **terminal** is the window or app where you type text commands.
- The **shell** is the program running inside that terminal and interpreting your commands.
- Closing a **terminal** usually ends the shell session that was running inside it.
- The **prompt** is the text shown before your cursor when the shell is ready.
- Prompts often include your username, host, current folder, or a symbol such as dollar sign.
- You type the command after the **prompt**, but the prompt itself is not part of the command.

Examples

```
# Show terminal type
echo "$TERM"
# Show current shell
echo "$SHELL"
# Type after the prompt
whoami
# Show prompt variable
echo "$PS1"
```

1.3 Interactive Shell Versus Script · Command Shape

- An **interactive** shell runs one command at a time as you type it.
- A **script** is a file of commands that Bash runs from top to bottom.
- The **same** command can often be tested interactively before it is saved into a script.
- Most shell commands start with a **command** name followed by options and arguments.
- Options usually change behavior, while arguments usually name files, folders, or values.
- Spaces separate each word, so the shell can tell where one part ends and the next begins.

Examples

```
# Run interactively
echo "Testing one command"
# Run a script file
bash hello.sh
# Command with option
ls -l
# Command with argument
mkdir practice
```

1.4 Tab Completion

- **Tab completion** asks the shell to finish a command name, file name, or folder name.
- It saves typing and helps avoid spelling mistakes in long paths.
- Pressing **Tab** twice can show possible matches when more than one completion exists.

Examples

```
# Complete a directory
cd Doc<Tab>
# Complete a command
hos<Tab>
```

2 Section 2

Navigation and Paths

2.1 pwd · Absolute Paths

- The working directory is the folder your shell is currently using.
- The **pwd** command prints that folder as a full path.
- Checking **pwd** helps you confirm where commands will create, read, or remove files.
- An **absolute** path starts at the root directory and names every folder needed to reach a file.
- On Linux, **absolute paths** begin with a slash.
- **Absolute paths** work the same no matter which directory you are currently in.

Examples

```
# Show current path
pwd
# Print path in a message
echo "You are in: $(pwd)"
# List an absolute path
ls /tmp
# Change to home by path
cd "$HOME"
```

2.2 Relative Paths · cd

- A **relative** path starts from your current working directory.
- **Relative paths** are shorter when you are already near the files you want.
- A **relative** path can break if you run the same command from a different folder.
- The **cd** command changes your current working directory.
- Running **cd** with no argument returns you to your home directory.
- The tilde character is another short way to refer to your home directory.

Examples

```
# List a child folder
ls scripts
# Open a nearby file
cat notes.txt
# Return home
cd
# Use tilde home
cd ~
```

2.3 cd .. · cd

- Two dots mean the parent directory of the current directory.
- Using **cd ..** moves one level upward in the folder tree.
- You can chain parent references when you need to move up more than one level.
- A single dash after **cd** means the previous working directory.
- **This** is useful when you need to jump between two folders repeatedly.
- Bash prints the destination path after a successful **cd** - command.

Examples

```
# Go up one level
cd ..
# Go up two levels
cd ../../
# Switch back
cd -
# Visit and return
cd /tmp
cd -
```

2.4 Working Directory in Scripts

- A script usually starts in the directory where you launched it, not where the script file lives.
- Commands inside the script use that starting directory unless the script changes it.
- Printing `pwd` inside a script is a simple way to understand where it is running.

Examples

```
# Show script location effect
printf "cwd=%s\n" "$(pwd)"
# Run from another folder
cd /tmp
bash /path/to/script.sh
```

3 Section 3

Listing and Inspecting Directories

3.1 ls · ls -l

- The **ls** command lists files and folders in a directory.
- Without arguments, **ls** lists the current working directory.
- You can give **ls** a path when you want to inspect a different directory.
- The **-l** option shows one item per line with extra details.
- Long listings include permissions, owner, size, and modification time.
- Use long listings when names alone are not enough information.

Examples

```
# List current directory
ls
# List another directory
ls /tmp
# Show details
ls -l
# Show details for one path
ls -l notes.txt
```

3.2 ls -a · ls -h

- Hidden file names begin with a dot on Unix-like systems.
- The -a option includes hidden files in the listing.
- Many configuration files are hidden so normal folder views stay less crowded.
- The -h option makes file sizes easier for people to read.
- **It** is most useful together with -l because long listings display sizes.
- Human-readable sizes use units such as K, M, and G.

Examples

```
# Include hidden files
ls -a
# Long hidden listing
ls -la
# Long human listing
ls -lh
# All details with sizes
ls -lah
```

3.3 tree · File Type Characters

- The **tree** command shows folders and files as a visual hierarchy.
- It helps beginners understand how a project is arranged.
- Some minimal systems do not include **tree** until it is installed.
- The first character in `ls -l` output describes the kind of item.
- A dash usually means a regular **file**, `d` means directory, and `l` means symbolic link.
- Recognizing these characters helps you choose safe commands for each item.

Examples

```
# Show current tree
tree
# Limit tree depth
tree -L 2
# See type characters
ls -l
# Show directory entry
ls -ld .
```

3.4 Permissions Overview

- **Permissions** describe who can read, write, or execute a file.
- The letters r, w, and x stand for read, write, and execute.
- Directories need execute permission so you can enter or search them.

Examples

```
# View permissions
ls -l script.sh
# View directory permissions
ls -ld .
```

4 Section 4

Creating and Moving Files



4.1 mkdir · mkdir -p

- The **mkdir** command creates a new directory.
- **It** is commonly used to prepare a workspace before creating files.
- A clear directory name makes later commands easier to understand.
- The -p option creates missing parent directories along the path.
- It also avoids an error when the target directory already exists.
- **This** is useful in scripts that may run more than once.

Examples

```
# Create one directory
mkdir lab
# Create two directories
mkdir input output
# Create nested folders
mkdir -p lab/raw
# Create repeatable output
mkdir -p reports/daily
```

4.2 touch · cp

- The **touch** command creates an empty file when the name does not exist.
- If the file already exists, **touch** updates its modification time.
- Beginners often use **touch** to make practice files quickly.
- The **cp** command copies a file to a new name or location.
- Copying keeps the original file in place.
- Check the destination path before copying over an existing file.

Examples

```
# Create one file
touch notes.txt
# Create several files
touch one.txt two.txt
# Copy with new name
cp notes.txt notes.bak
# Copy into directory
cp notes.txt lab/
```

4.3 cp -r · mv

- Directories contain other entries, so **cp** needs recursive mode to copy them.
- The **-r** option copies a directory and its contents.
- Recursive copying can duplicate a lot of data, so choose destinations carefully.
- The **mv** command can rename a file in the same directory.
- Renaming changes the name but does not copy the file contents.
- A rename can replace an existing destination if you are not careful.

Examples

```
# Copy a directory
cp -r lab lab-backup
# Copy into backups
cp -r reports backups/
# Rename a file
mv draft.txt final.txt
# Rename a directory
mv lab practice-lab
```

4.4 mv - Move a File

- The **mv** command also moves files or directories to another location.
- Moving changes where the item lives in the directory tree.
- Use a trailing slash on an existing directory when you want to move into it.

Examples

```
# Move file into folder
mv final.txt reports/
# Move directory into folder
mv practice-lab archive/
```

5 Section 5

Removing and Viewing Text

5.1 rm · rm -r

- The **rm** command deletes files.
- Deleted files are not normally moved to a desktop trash folder.
- Use **rm** carefully and list files first when you are uncertain.
- The **-r** option removes a directory and everything inside it.
- Recursive removal is powerful and can delete many files quickly.
- Confirm the path before running **rm -r**, especially near your home directory.

Examples

```
# Remove one file
rm old.txt
# Preview then remove
ls old.txt
rm old.txt
# Remove a practice folder
rm -r lab
# Remove generated reports
rm -r reports/tmp
```

5.2 cat · less

- The **cat** command prints file contents directly to the terminal.
- **It** is best for short files that fit comfortably on one screen.
- For long files, a pager such as **less** is easier to control.
- The **less** command opens text in a scrollable pager.
- You can move with arrow keys, Page Up, and Page Down.
- Press q to quit **less** and return to the shell prompt.

Examples

```
# Show a text file
cat notes.txt
# Combine two files
cat header.txt body.txt
# Open a long file
less system.log
# Read command output
ls -la | less
```

5.3 more · head

- The **more** command is an older pager that displays text page by page.
- **It** is less flexible than **less** but still appears on many systems.
- Knowing **more** helps when you work on a small or older Unix environment.
- The **head** command prints the beginning of a file.
- By default, it shows the first ten lines.
- Use **-n** when you want a different number of lines.

Examples

```
# Page through a file
more README.txt
# Page through listing
ls -la | more
# Show first lines
head app.log
# Show first five lines
head -n 5 app.log
```

5.4 tail - Last Lines

- The **tail** command prints the end of a file.
- **It** is often used to inspect the newest entries in a log file.
- The -n option controls how many ending lines are displayed.

Examples

```
# Show last lines  
tail app.log  
# Show last twenty lines  
tail -n 20 app.log
```

6 Section 6

Finding Commands and Getting Help

6.1 which · type

- The **which** command searches PATH and prints the executable it would run.
- **It** is useful when you want to know where a command comes from.
- Shell builtins and aliases may not always be explained fully by **which**.
- The **type** command tells Bash how it understands a command name.
- It can identify builtins, aliases, functions, and executable files.
- Use type when **which** does not explain the whole story.

Examples

```
# Find Bash
which bash
# Find Python
which python3
# Explain cd
type cd
# Explain ls
type ls
```

6.2 command -v · man

- **The** command -v form asks the shell to locate a command in a script-friendly way.
- It works well when a script needs to check whether a tool is available.
- A successful lookup prints the **command** path or name and returns success.
- The **man** command opens the system manual for many commands.
- Manual pages describe options, arguments, behavior, and related commands.
- Press q to leave the manual page when you are finished reading.

Examples

```
# Check for sed
command -v sed
# Use in a condition
command -v git >/dev/null
# Open ls manual
man ls
# Open Bash manual
man bash
```

6.3 --help · history

- Many commands accept **--help** to print a short usage summary.
- Help output is usually faster to scan than a full manual page.
- Not every command supports the exact same help option style.
- The **history** command lists commands you typed earlier in the shell.
- **History** helps you repeat, review, and learn from previous work.
- Be careful because **history** may include paths or values you typed by mistake.

Examples

```
# Show mkdir help
mkdir --help
# Show cp help
cp --help
# Show recent history
history
# Show last five entries
history 5
```

6.4 clear - Clean the Screen

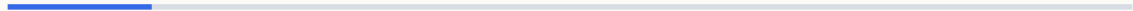
- The **clear** command redraws the terminal with an empty view.
- It does not delete files or erase command history.
- Clearing the screen can make a new practice step easier to see.

Examples

```
# Clear terminal
clear
# Print after clear
clear
echo "Ready"
```

7 Section 7

Output and First Scripts



7.1 echo · printf

- The **echo** command prints text followed by a newline.
- **It** is useful for quick messages and simple checks.
- For precise formatting in scripts, **printf** is usually a better choice.
- The **printf** command prints text using a format string.
- It does not add a newline unless the format asks for one.
- **Printf** behaves more predictably than echo across different shells.

Examples

```
# Print a greeting
echo "Hello"
# Print two words
echo "Bash basics"
# Print with newline
printf "Hello\n"
# Print a value
printf "User: %s\n" "$USER"
```

7.2 Comments in Bash · exit

- A comment explains code for humans and is ignored by Bash.
- **Comments** begin with a hash character when it is not inside quotes.
- Good **comments** explain why a command exists, not just what the command name says.
- The **exit** command ends the current shell or script.
- A numeric **exit** status can tell other programs whether the script succeeded.
- By convention, zero means success and nonzero means something failed.

Examples

```
# Single comment
# Create the output directory
mkdir -p output
# Comment after command
date # Show the current date
# Exit successfully
exit 0
# Exit with failure
exit 1
```

7.3 Shebang for Bash Scripts · chmod +x

- A **shebang** is the first line that tells the system which interpreter should run a script.
- The form using `env` finds Bash through the current `PATH`.
- A **shebang** matters when you run the script directly as a program.
- The **chmod** command changes file permissions.
- Adding `+x` gives permission to execute a script file.
- Executable permission is needed when you run a script with `./script.sh`.

Examples

```
# Bash shebang
#!/usr/bin/env bash
# First script lines
#!/usr/bin/env bash
echo "Hello"
# Make executable
chmod +x hello.sh
# Check permission
ls -l hello.sh
```

7.4 bash Script Versus ./script

- Running bash **script**.sh asks Bash to read the file even if it is not executable.
- Running ./script.sh asks the system to execute the file directly.
- Direct execution needs both a usable shebang and execute permission.

Examples

```
# Run with Bash
bash hello.sh
# Run directly
./hello.sh
```

8 Section 8

Environment and Identity

8.1 PATH Explained · PATH Search Order

- **PATH** is a list of directories the shell searches for **Examples** command names.
- When you type `ls`, Bash looks through **PATH** to find the program or builtin behavior.
- Understanding **PATH** helps explain command not found errors.
- Bash checks **PATH** directories from left to right.
- If two directories contain the same command name, the earlier directory wins.
- Changing **PATH** order can change which program runs.

```
# Print PATH
echo "$PATH"
# Split PATH lines
printf "%s\n" "$PATH"
# Show command chosen
command -v python3
# Show PATH value
echo "$PATH"
```

8.2 env · printenv

- The **env** command prints environment variables passed to programs.
- Environment variables are name and value pairs available to child processes.
- The output may be long, so reading it through a pager can help.
- The **printenv** command prints all environment variables or one selected variable.
- **It** is a clear way to check values such as HOME, USER, and PATH.
- If a requested variable is missing, **printenv** usually prints nothing.

Examples

```
# Print environment
env
# Page environment
env | less
# Print HOME
printenv HOME
# Print all variables
printenv
```

8.3 export · whoami

- The **export** command marks a shell variable to be passed to child programs.
- **It** is often used when a script or tool reads configuration from the environment.
- Exported values affect commands started after the **export** command.
- The **whoami** command prints the username of the current effective user.
- It helps you confirm whether you are working as yourself or another account.
- User identity matters for file ownership and permissions.

Examples

```
# Export one value
export APP_ENV=dev
# Use exported value
bash -c 'echo "$APP_ENV"'
# Show current user
whoami
# Use in a message
echo "Running as $(whoami)"
```

8.4 hostname - Current Machine Name

- The **hostname** command prints the name of the computer or container.
- It helps when you work across multiple servers or remote sessions.
- Including the **hostname** in logs can make troubleshooting easier.

Examples

```
# Show host name
hostname
# Print with user
echo "$(whoami)@$(hostname)"
```

9 Section 9

System Awareness and Patterns

9.1 date · df

- The **date** command prints the current date and time.
- It can also format timestamps for filenames and logs.
- Using timestamps helps separate repeated script runs.
- The **df** command reports free and used space on mounted filesystems.
- The **-h** option makes the sizes easier to read.
- Checking **df** helps diagnose failures caused by a full disk.

Examples

```
# Show date
date
# Create compact date
date +"%Y%m%d"
# Show disk space
df -h
# Check current filesystem
df -h .
```

9.2 du · find

- The **du** command estimates how much disk space files and directories use.
- The -s option summarizes instead of listing every child path.
- The -h option prints human-readable sizes.
- The **find** command walks directories and matches files by rules you choose.
- It can search by name, type, size, time, and many other properties.
- Start **find** in a small directory while you are learning its options.

Examples

```
# Summarize current folder
du -sh .
# Summarize directories
du -sh */
# Find text files
find . -name "*.txt"
# Find directories
find . -type d
```

9.3 locate Versus find · Wildcard Star

- The **locate** command searches a prebuilt database of file paths.
- **Locate** can be faster than find, but its database may be out of date.
- Find searches the live filesystem, so it sees current files immediately.
- The star **wildcard** matches zero or more characters in a file name.
- The shell expands the **wildcard** before the command receives the arguments.
- Use star patterns carefully so you know which files a command will touch.

Examples

```
# Use locate
locate bashrc
# Use live search
find . -name ".bashrc"
# List text files
ls *.txt
# Remove backup files
rm *.bak
```

9.4 Wildcard Question and Brackets

- The question mark **wildcard** matches exactly one character.
- Bracket patterns match one character from a selected set or range.
- These patterns are useful when file names follow a predictable structure.

Examples

```
# Match one digit
ls file?.txt
# Match a range
ls report-[1-3].txt
```

10

Section 10

Safe Command Habits

10.1 Paths with Spaces · Case-Sensitive Names

- Spaces separate words in shell commands.
- A file name with spaces must be quoted so the shell treats it as one path.
- Quoting **paths** is a simple habit that prevents many beginner mistakes.
- Linux file names are usually **case-sensitive**.
- Notes.txt and notes.txt can be two different files.
- Matching the exact capitalization prevents confusing file not found errors.

Examples

```
# Create spaced file
touch "my notes.txt"
# List spaced file
ls -l "my notes.txt"
# Create two names
touch Notes.txt notes.txt
# List both names
ls *otes.txt
```

10.2 Preview Before Destructive Commands · --

- Destructive commands change or remove data. **Examples**
- Previewing paths with `ls` or `find` helps confirm your target before you delete.
- This habit is especially important when wildcards or recursive options are involved.
- A double dash tells many commands that options are finished.
- It helps when a file name begins with a dash and could look like an option.
- This pattern is common in safer scripts that handle unusual names.

```
# Preview matches
ls *.tmp
# Delete after preview
rm *.tmp
# Remove dashed name
rm -- -draft.txt
# List dashed name
ls -- -draft.txt
```

10.3 Command Not Found · Permission Denied

- **Command not found** means Bash could not locate what you typed as a command.
- The cause may be a spelling mistake, a missing program, or a PATH issue.
- **Use** command -v and check spelling before changing your environment.
- **Permission denied** means the system refused the requested access.
- For scripts, the common cause is missing execute **permission**.
- For files and folders, inspect ownership and permissions with ls -l.

Examples

```
# Check command
command -v tree
# Inspect PATH
echo "$PATH"
# Inspect a script
ls -l run.sh
# Add execute permission
chmod +x run.sh
```

10.4 Read Error Messages

- Error messages often name the command, path, and kind of problem.
- Reading the first error carefully can save a lot of guessing.
- Run a smaller command when a long command produces confusing output.

Examples

```
# Trigger missing file
cat missing.txt
# Check current path
pwd
```

11

Section 11

Keyboard Workflow

11.1 Ctrl-C · Ctrl-D

- **Ctrl-C** sends an interrupt signal to the running foreground command.
- **It** is the usual way to stop a command that is taking too long.
- After the command stops, the shell prompt returns.
- **Ctrl-D** tells the terminal there is no more input to read.
- At an empty shell prompt, it usually exits the shell.
- Inside programs that read input, it can signal end of file.

Examples

```
# Start a wait
sleep 30
# Interrupt it
Press Ctrl-C
# Exit an empty prompt
Press Ctrl-D
# End cat input
cat
Press Ctrl-D
```

11.2 Ctrl-L · History Navigation Keys

- **Ctrl-L** clears the terminal view like the clear command.
- It keeps the current command line available while cleaning the screen.
- This shortcut is useful during demonstrations and practice labs.
- The up and down arrow keys move through recent commands.
- Reusing **history** saves typing and reduces spelling mistakes.
- Always review a recalled command before pressing Enter.

Examples

```
# Clear by shortcut
Press Ctrl-L
# Clear by command
clear
# Recall previous command
Press Up
# Run history list
history 10
```

11.3 Editing the Command Line · Aliases Intro

- **Basic** command-line editing lets you fix a command before running it.
- Left and right arrows move the cursor within the current command.
- Backspace and Delete remove characters around the cursor.
- An **alias** is a short name that expands to another command.
- **Aliases** are convenient for interactive shortcuts.
- Scripts should usually use full commands so their behavior is clearer.

Examples

```
# Move within command
Press Left or Right
# Fix then run
echo "helo"
# edit to hello
# Create an alias
alias ll='ls -la'
# Use the alias
ll
```

11.4 Removing an Alias

- The **unalias** command removes an alias from the current shell session.
- **Removing an alias** helps when a shortcut hides the real command behavior.
- Aliases defined in startup files may return in a new terminal.

Examples

```
# Remove one alias
unalias ll
# List aliases
alias
```

1 2

Section 12

Practice Lab and Checkpoints

12.1 Create a Practice Lab · Make Practice Files

- A practice lab is a safe folder where you can try commands freely.
- Keeping practice work separate lowers the risk of changing important files.
- A fresh lab makes it easier to repeat examples from the beginning.
- Small practice files let you test copy, move, view, and remove commands.
- Using predictable names makes wildcard behavior easier to see.
- You can recreate practice files whenever you need another clean run.

Examples

```
# Create lab folders
mkdir -p bash-lab/input bash-lab/output
# Enter the lab
cd bash-lab
# Create sample files
touch input/a.txt input/b.txt
# Add sample text
printf "one\ntwo\n" > input/a.txt
```

12.2 Hidden Configuration Files · File Naming Habits

- Many shell settings live in **hidden** files in your home directory.
- Examples include files such as `.bashrc` and `.profile`.
- List **hidden** files before editing them so you know what already exists.
- Simple names make shell commands easier to read and type.
- Lowercase letters, numbers, dashes, and underscores are usually beginner-friendly.
- Avoiding spaces in practice names reduces quoting distractions while learning basics.

Examples

```
# List hidden home files
ls -la ~
# Check bashrc
ls -l ~/.bashrc
# Create simple names
touch report-01.txt
# Create underscore name
mkdir daily_reports
```

12.3 Checkpoint

- A navigation **checkpoint** confirms that you can move around safely.
- You should be able to print your location, move to a parent folder, and return home.
- Practice until you can predict how `pwd` changes after each `cd` command.
- A file operation **checkpoint** confirms that you can create, copy, move, and remove items.
- Use only practice files until each command feels predictable.
- List the directory after each step so you can see what changed.

Examples

```
# Run navigation check
pwd
cd ..
# ...
# Return home and check
cd
pwd
touch a.txt
cp a.txt b.txt
mv b.txt output.txt
rm output.txt
```

12.4 Checkpoint - Help and Discovery

- A help **checkpoint** confirms that you can discover commands without memorizing everything.
- Use `type` or command `-v` to identify commands before reading deeper help.
- Use `man` or `--help` when you need options, examples, or exact behavior.

Examples

```
# Identify a command
type printf
# Open quick help
printf --help
```

NEXT STEP

Practice every example in your terminal

Then open the next module deck

Merit Advisory

01 · Shell Basics

